

GWDC Korea Hackathon: FuriosaAI X Bricksum ·Agent Finance Bonus Track Challenge Brief

Challenge A: Build a Financial Service Powered by AI Agents and Blockchain

Build a financial service prototype in which an AI agent helps users complete a purchase, payment, or asset management task, prevents spending outside the user's instructions, or helps investigate a transaction dispute. Use the NPU-based Kiln API, an LLM token API, to power the agent and integrate blockchain into the selected workflow. Kiln provides access to low-power NPU inference, enabling teams to explore how efficiently a complete service can operate in terms of token usage and energy consumption. Connect the user's request and relevant inputs, such as a budget, permitted items or merchants, and a deadline, to actions and records that can be inspected. Possible implementations include a purchasing agent that respects user conditions, a wallet interface for approving and stopping agent spending, buyer and seller agents that negotiate and settle a transaction, or a tool that explains purchases using approval and transaction records. Teams may focus on one function, combine several, or propose another idea within agent finance.

Acceptance Criteria

- **User Need & Workflow:** Identify the intended user and the problem being solved. Demonstrate the selected workflow from user input to a usable outcome, and explain which task the AI agent performs and which parts you kept in code.
- **Kiln API Integration & Efficiency:** The resulting AI agent must operate using the NPU-based Kiln API with gpt-oss-120b. Demonstrate actual API calls within the selected workflow and show how the responses inform the agent's decisions or actions. Report token usage broken down by flow rather than as a single total, and explain how the design reduces unnecessary inference and energy consumption, supporting energy estimates with available measurements or clearly stated assumptions.
- **Blockchain Integration:** Demonstrate the selected functionality on a devnet or testnet. An end-to-end run of the workflow should produce at least one on-chain transaction — a payment, a settlement, or a record written on-chain — and provide its transaction hash with the matching log or history entry. Show how the chain is used by the workflow: which state the agent reads, writes, or settles.
- **Selected Function:** Declare in one sentence, in your README, the function you built — a purchasing agent, a wallet and approval tool, negotiating buyer and seller agents, an evidence and dispute tool, or another function within agent finance. The work is evaluated against the function you declared rather than a fixed list of expected features.
- **Condition Checks & Evidence:** Repeat the end-to-end run twice with the user's conditions changed — for example a reduced budget, or a merchant that is no longer permitted — and verify the behavior through logs or history. Spending controls should stop actions outside the permitted scope. Evidence or dispute tools should let another person inspect the records and assess whether the transaction followed the user's conditions.

Challenge B: Build the Controls and Records for an AI Agent That Spends

Build a financial service prototype for the moment after money has been delegated to an AI agent: what keeps its spending inside the line the user drew, and what settles the question afterwards when something goes wrong. Use the NPU-based Kiln API, an LLM token API, to power the agent, and integrate blockchain into the workflow you choose. Kiln provides access to low-power NPU inference, enabling teams to explore how efficiently a complete service can operate in terms of token usage and energy consumption. Payment rails record who paid whom, but not who authorized it or under what conditions, so an agent that stays within a budget does so only because it was built to. Possible implementations include a wallet interface for approving, watching, and stopping agent spending, a policy layer that holds a budget and a permitted list across a whole session, a tool that explains a past purchase using approval and transaction records, or another way of making delegated spending answerable.

Acceptance Criteria

- **Declared Function & User Need:** Declare in one sentence, in your README, the function you built. Identify the intended user and the problem being solved, demonstrate the workflow from user input to a usable outcome, and explain which task the AI agent performs and which parts you kept in code. The work is evaluated against the function you declared rather than a fixed list of expected features.
- **Boundaries & Stopping:** State the boundary the agent must not cross and where in your system it is enforced. Include at least two runs in which the agent is pushed outside the permitted scope — a budget exceeded once fees are added, a merchant that is not on the list, a deadline already past — and show through logs or history that it stopped. Stopping is a correct outcome, and it should be recorded rather than silent.
- **Kiln API Integration & Efficiency:** The resulting AI agent must operate using the NPU-based Kiln API with gpt-oss-120b. Demonstrate actual API calls within the selected workflow and show how the responses inform the agent's decisions or actions. Report token usage broken down by flow rather than as a single total, and explain how the design reduces unnecessary inference and energy consumption, supporting energy estimates with available measurements or clearly stated assumptions.
- **Blockchain Integration:** Demonstrate the selected functionality on a devnet or testnet. An end-to-end run of the workflow should produce at least one on-chain transaction — a payment, a settlement, or a record written on-chain — and provide its transaction hash with the matching log or history entry. Show how the chain is used by the workflow: which state the agent reads, writes, or settles.
- **Approval & Evidence:** Show the human side of the workflow: how a person grants a budget, follows what is being spent, stops the agent, and receives a receipt. Then show that another person, working from your records alone, can reconstruct whether a completed payment was inside what the user allowed.